GoServer

Project team

October 26, 2026

Contents

1 Project sketch **1**
- 1.1 People and systems . . . 1
- 1.2 Primary navigation . . . 1
- 1.3 Initial game format . . . 2
- 1.4 Teams and alliances . . . 2
- 1.5 Bots . . . 3
- 1.6 Chat . . . 3
- 1.7 Representative scenarios . . . 3
  - 1.7.1 Find and enter a game . . . 3
  - 1.7.2 Play a timed move . . . 3
  - 1.7.3 Capture neighboring groups . . . 3
  - 1.7.4 Play a suicide move . . . 3
  - 1.7.5 Fill a seat with a bot . . . 4
  - 1.7.6 Form an alliance . . . 4
  - 1.7.7 Score a team . . . 4
  - 1.7.8 Chat inside a game . . . 4

2 Requirements argument **5**
- 2.1 Observable scope . . . 5
- 2.2 Proposed safety and consistency invariants . . . 5
- 2.3 Resolved capture and suicide ordering . . . 5
- 2.4 Resolved scoring direction . . . 6
- 2.5 Resolved bot execution direction . . . 6
- 2.6 Recovered ko and repetition behavior . . . 6
- 2.7 Open rules decisions . . . 7

3 Implementation argument **8**

4 Current reduced server-only scope **9**

5 Room creation and working browser revision **10**

6 N-player Go play revision **12**

7 Lab parity revision: seats, teams, history and search **14**
- 7.1 Request . . . 14
- 7.2 Decisions recorded from the stakeholder . . . 14
- 7.3 Derived rules . . . 14

7.4 What every viewer sees . . . 15
7.5 Protocol shape . . . 15
7.6 Deferred . . . 16
7.7 Implementation argument . . . 16

8 Usage scenarios **18**
8.1 Welcome . . . 18
8.2 Creating a challenge . . . 19
8.3 Planning seats . . . 20
8.4 Reading the bot's search . . . 21
8.5 A watcher explores a variation . . . 22
8.6 Proposing a team merge . . . 23
8.7 Playing as a team, on autopilot . . . 24
8.8 The result . . . 25

Chapter 1

Project sketch

GoServer is a KGS-inspired online Go service focused on multiplayer variants rather than direct competition with established two-player Go servers. The initial release centers on short, four-participant games played on an 11×11 board.

KGS provides inspiration for the navigation model, but this sketch does not require compatibility with KGS accounts, protocols, clients, or server internals.

1.1 People and systems

The primary participant is a signed-in person who wants to browse chat rooms, find or observe games, play Go, and converse with other participants. A game seat may instead be controlled by a server-supported bot. People and bots are both game participants for the purposes of turn order and team membership.

The server maintains identities, sessions, rooms, games, board state, clocks, team membership, participant graveyards, and chat history. Connected clients present server-authoritative state and submit participant actions.

1.2 Primary navigation

After a successful login, a person sees the available chat rooms. Entering a room displays the room conversation and the games associated with that room. Entering a game displays its board, participants, teams, move deadline, current turn, game status, and game-specific chat.

The intended hierarchy is:

1. authenticate;
2. browse and enter a chat room;
3. inspect the games associated with that room;
4. enter a game as a player or observer; and
5. play or observe while using the game chat.

Room chat and game chat are separate conversations. A message sent to one is not implicitly copied to the other.

1.3 Initial game format

The first supported format has the following fixed profile:

- an 11×11 Go board;
- four participant seats;
- sequential turns;
- a ten-second deadline for each move;
- each seat configurable as either a person or a bot; and
- each participant initially occupying a distinct team.

The server is authoritative for legal moves, turn order, deadlines, captures, repetition rules, team membership, graveyard contents, and the final result. Clients must not resolve conflicting game actions locally.

When a stone is placed, the server first examines its orthogonally adjacent groups. Every adjacent group that has no remaining liberties is removed before the server evaluates the placed stone for suicide. This ordering permits an otherwise surrounded placement to survive when it first captures neighboring stones and thereby gains liberties.

If no neighboring group is removed and the placed stone's connected same-owner group has no liberties, the move is a permitted suicide move. The server removes the newly placed stone and its entire connected same-owner group. This permits multiplayer tactics in which a participant removes their own group to help an allied participant form eyes. Every self-removed stone is placed in its owner's graveyard, just like a stone removed by another participant.

1.4 Teams and alliances

Every participant begins the game as the sole member of a separate team. Participants may subsequently join together so that multiple participants belong to one team. A team therefore represents an alliance of one or more seats rather than a single shared seat.

A team merge changes cooperation and scoring without removing a participant's seat or turn. Each participant continues to place stones during that participant's assigned turns. Existing stones remain on the board when teams merge.

For each participant, the server maintains a graveyard containing that participant's stones that have been removed from play. When, for example, White captures a Black stone, the removed stone is given to the Black participant's graveyard rather than credited to White. A stone removed by its owner's suicide is likewise placed in that owner's graveyard. Removal therefore does not directly award points to another participant.

A team's score is its total territory minus its total losses. Total losses are the number of removed stones in the graveyards of the team's members. Equivalently, if A_T is the territory attributed to team T and G_T is the combined number of stones in its members' graveyards, then

$$S_T = A_T - G_T.$$

This makes the result express territory gained at the cost of stones lost. The approval, timing, and reversibility of a team merge remain requirements decisions.

1.5 Bots

Each of the four seats can be assigned independently to a person or bot. This permits any mixture from four people to four bots. Computer-controlled seats use Monte Carlo tree search executed by the server. Clients do not run the authoritative bot search or submit bot-selected moves on behalf of a bot seat.

A bot is subject to the same legal-move checks and ten-second move deadline as a person. Its server-side search must stop when its turn expires, and a result that arrives after the authoritative deadline must not advance the game. Bot failures must not give the bot permission to bypass the clock, acquire privileges unavailable to its assigned seat, or submit moves after its turn has ended. The implementation must bound and isolate bot computation so that one search cannot exhaust resources needed by unrelated games.

1.6 Chat

A participant can converse in a room without entering a game. Every game also has a dedicated chat visible to people who have entered that game, subject to the eventual observer and moderation policy. Bot participation in chat is not required by the initial sketch.

1.7 Representative scenarios

1.7.1 Find and enter a game

A person logs in, sees the room list, enters a room, and receives the current room conversation and game list. The person selects a game and receives its board, participants, teams, clock state, graveyard counts, and game chat.

1.7.2 Play a timed move

When a participant's turn begins, the server starts a ten-second deadline. The participant submits a board coordinate. The server accepts the move only if the participant controls the active seat, the turn is still current, the deadline has not expired, and the move is legal. The accepted state is then distributed to everyone viewing the game.

1.7.3 Capture neighboring groups

A participant places a stone next to one or more groups that have lost their final liberty. The server removes all such neighboring groups before evaluating whether the newly placed stone's group has liberties. Each removed stone is placed in the graveyard of the participant who owned it. No capture point is awarded to the participant who placed the stone.

1.7.4 Play a suicide move

A participant places a stone that does not remove any neighboring group, and the placed stone's connected same-owner group is left without liberties. The server treats the move as a permitted suicide and removes the placed stone together with its entire connected same-owner group. Every self-removed stone is placed in its owner's graveyard and therefore contributes to that owner's losses.

1.7.5 Fill a seat with a bot

A game is configured with one or more bot-controlled seats. When a bot-controlled turn begins, the server runs that bot's Monte Carlo tree search within server-controlled execution. The search operates under the same ten-second deadline and move-legality constraints as a human-controlled seat. Only a result selected before the authoritative deadline may be submitted for the current turn.

1.7.6 Form an alliance

Participants who began on separate teams request to become members of the same team. The server applies the selected merge policy, records the resulting membership, and distributes the updated team state to all viewers. Existing stones remain on the board. The resulting team's loss total includes the graveyards of all its members.

1.7.7 Score a team

At the end of a game, the server totals the territory attributed to a team and the removed stones held in its members' graveyards. It subtracts the graveyard total from the territory total and reports the resulting team score. Captures and suicides affect scoring only as losses borne by the owners of the removed stones.

1.7.8 Chat inside a game

A person viewing a game sends a message to the game chat. The server associates the message with that game and distributes it to authorized game viewers without adding it to the containing room's chat.

Chapter 2

Requirements argument

2.1 Observable scope

The candidate first-release requirements will cover authentication, room discovery, room entry, room chat, room-associated game discovery, game entry, game chat, four-seat game setup, human and bot seats, server-executed bot search, server-authoritative timed turns, multiplayer Go rules, participant graveyards, team merges, scoring, reconnect behavior, and observable error responses.

Account registration, moderation administration, persistent ratings, tournaments, KGS protocol compatibility, and sophisticated bot strategy beyond the required server-side Monte Carlo tree search are not yet part of the committed first-release scope.

2.2 Proposed safety and consistency invariants

The requirements should preserve the following positive invariant: every externally visible room, chat, game, move, clock, capture, graveyard, score, and team transition is authorized for the acting identity and derived from one unambiguous server-authoritative state.

Candidate forbidden behaviors include accepting a move from the wrong seat, accepting more than one move for a turn, accepting a move after its deadline, resolving suicide before neighboring captures, removing less or more than the entire connected same-owner group during suicide, crediting a removed stone to another participant, placing a removed stone in the wrong owner's graveyard, allowing a client to manufacture team membership or graveyard contents, accepting a client-supplied move as the output of a server-controlled bot, leaking private session credentials into chat or logs, crossing room or game boundaries through an identifier substitution, letting a bot acquire privileges unavailable to its assigned seat, and allowing one bot search to consume unbounded server resources.

These statements are design candidates rather than accepted requirements until represented by the typed project specification and explicitly accepted.

2.3 Resolved capture and suicide ordering

After a stone is placed, adjacent groups without liberties are removed first. Only if no neighboring group is removed does the server evaluate the placement as suicide. Thus a placement that captures a neighboring group can gain the liberties created by that removal and remain on the board.

Suicide is permitted as a multiplayer tactic, including to help an ally form eyes. If the placed stone's connected same-owner group has no liberties, the server removes the placed stone and that

entire connected group. All self-removed stones enter their respective owners' graveyards and count as losses in exactly the same way as stones removed by another participant.

2.4 Resolved scoring direction

A removed stone is assigned to the graveyard of the participant who owned it. It counts as one loss for that participant and is not a positive capture point for another participant.

A team's final score is its total territory minus the combined number of stones in its members' graveyards. Graveyard ownership remains attached to the participant, allowing team losses to be recomputed from authoritative participant records after a team merge. The typed specification must still define how territory is identified and attributed.

2.5 Resolved bot execution direction

Computer-player Monte Carlo tree search runs on the server rather than in a connected client. The authoritative game clock bounds the search, and a late search result cannot advance an expired or superseded turn. The implementation mapping must define resource bounds and execution isolation for simultaneous searches without changing the observable ten-second deadline.

2.6 Recovered ko and repetition behavior

The supplied N-Player Go Lab JavaScript provides concrete provenance for the intended ko behavior. Its user-facing description calls the rule "simple ko plus positional superko," but its state comparison includes both the board and the participant to move. The standard description of the implemented rule is therefore *situational superko*, not positional superko.

The candidate first-release baseline is as follows:

- the ko rule is enabled by default for a two-participant game and disabled by default for a game with three or more participants;
- when enabled, a qualifying single-stone capture establishes a simple-ko restriction that prevents the owner of the captured stone from immediately recapturing at that point on that participant's next turn;
- after every proposed stone placement, the resulting board together with the next participant to move must differ from every earlier such state on the current game line;
- passes, deadline expirations, and resignations are not rejected merely because they leave or recreate a repeated state; and
- independently of repetition, a game is capped at $3N^2$ moves for an $N \times N$ board.

The prototype represents a repetition state with a 32-bit Zobrist board hash combined with a token for the next participant. That representation is useful evidence of semantics but is not sufficient as the sole authoritative equality test because a hash collision could reject a legal position as repeated. The production requirement should demand collision-safe comparison, such as an exact canonical state representation or a digest backed by exact equality when hashes match.

The repetition check applies to actual game-tree branches and is mirrored inside Monte Carlo tree search: a searched stone placement is skipped when its resulting board-and-next-participant

state occurs in the real game history or in the current search path. This search behavior is advisory; the authoritative game transition must independently enforce the same observable rule.

2.7 Open rules decisions

The remaining core multiplayer rules still need durable answers for:

- whether the ko rule is fixed for the first-release four-participant format or configurable when a game is created;
- whether an expired deadline causes an automatic pass, forfeits the seat, or has another result;
- how territory is identified and attributed among participants and teams;
- when team merges are allowed, who must approve them, and whether they are reversible;
- whether observers may use game chat; and
- what reconnect grace, if any, exists within a ten-second turn.

Chapter 3

Implementation argument

Implementation is blocked until the observable behavior is encoded as a typed LeanFM requirement specification, checked, and explicitly accepted, followed by a checked and explicitly accepted implementation mapping.

Once those gates are satisfied, the implementation argument must map authentication, authorization, room isolation, game isolation, clocks, turn serialization, legal-move validation, ordered capture and suicide processing, graveyard attribution, collision-safe situational-superko validation, team transitions, server-side Monte Carlo tree search, bot resource isolation, persistence, chat delivery, and scoring attribution to concrete controls and tests. In particular, concurrent move and timeout processing must have a single atomic winner so that a deadline and a move cannot both advance the same turn. A bot result must additionally be bound to its game, seat, turn revision, and deadline so that a delayed or confused result cannot be applied to another turn. A placement, all neighboring removals, any resulting connected-group suicide removal, repetition validation, and all corresponding board, graveyard, and score-relevant changes must be committed as one authoritative transition.

Chapter 4

Current reduced server-only scope

The operator explicitly selected formalization and checking of the reduced basic-server slice in question `basic_server_requirements_gate`. The current requirements cover room discovery and entry, room game/challenge listings, named open challenge creation, watch/join navigation, room chat and per-game chat. A game is currently a navigable chat container. Earlier chapters' Go boards, move rules, bots, clocks, teams and scoring remain future roadmap material and are not obligations of this first server-only requirements gate.

Existing server-issued sessions identify callers. Joining a room or game establishes its chat membership; sender identity and timestamps come from the server. Chat is never implicitly copied across room/game boundaries. The draft adds request correlation and explicit rejection branches without choosing an implementation runtime or HTTP transport. Watch/join confers no Go move or seat authority in this slice. Account registration is not added.

Proposed reversible bounds are 128 characters for names, 4000 for chat text and the latest 100 chat entries per conversation. The retained 100-person population is a capacity forecast, not measured performance. Rejected or overloaded actions must not mutate state or disclose private identities or credentials. Replay and concurrency controls, connection limits, persistence and negative tests must be realized in the separate implementation mapping after requirements acceptance.

Canonical candidates are `spec/Requirements.lean` and `spec/Requirements.proto`. Checked task views are generated from that same specification into `spec/tasks.json`; the task catalog contains one interaction diagram and state machine for each scoped operation. No human gate is accepted by compilation or by preparing these files.

Chapter 5

Room creation and working browser revision

The October 25 request asks for working room creation and room-scoped game challenges, with placeholder links removed. This candidate extends the previous accepted seven-task server slice with `create_room`. A room name is a trimmed, case-sensitive, public display identifier of 1–128 Unicode characters. A valid local session may create it; duplicate names and the 16-room bound reject without mutation. The creator subsequently enters it with `EnterRoom`. Challenge results and listings retain their entered name. All state remains in memory and resets on restart. Gameplay remains deferred.

The positive invariant is that each displayed room, challenge and chat result comes from an authorized server operation in its selected scope. Forbidden outcomes include duplicate mutations on retry or concurrent admission, room or game chat disclosure outside membership, executable user text, stale responses replacing another room's content, exposed tokens, and navigation to fabricated destinations. The proposed controls and negative checks are cataloged in `reports/rooms-browser-review/review.md`. They are obligations for the separate implementation review, not claims of completed source changes.

The current typed requirements generate interaction and state-machine views in `spec/tasks.json`; extracted views and their exact provenance appear in `reports/rooms-browser-review/`. This revision requires explicit requirements acceptance followed by acceptance of its implementation mapping. Earlier protected gate snapshots remain evidence only for their earlier input revisions. Billing failure of the attempted Anthropic request did not accept a gate or implement this feature.

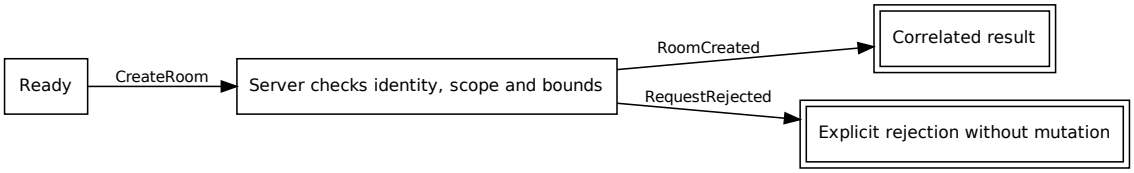


Figure 5.1: Create a named chat room: state transitions generated from the current typed task.

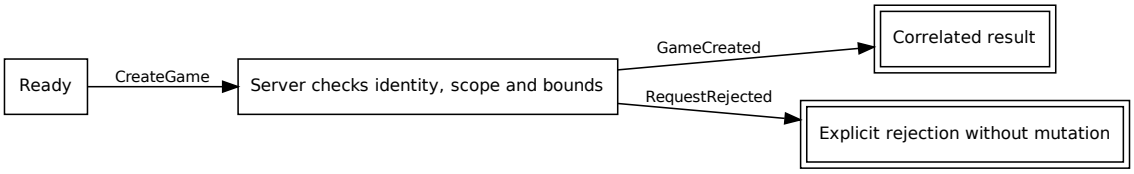


Figure 5.2: Create a named room game or challenge: state transitions generated from the current typed task.

Chapter 6

N-player Go play revision

The October 26 request asks to complete the KGS-style server: challenge creation takes the board parameters of the stakeholder's N-Player Go Lab, and seats not taken by people are played by Monte Carlo tree search bots running in the server. This candidate extends the twelve-task specification with `take_seat`, `start_game`, `play_move` and `game_state`, and changes `create_game` parameters. It answers the open rules decisions of the requirements argument as follows.

- **Parameters.** Board 5, 7, 9, 11, 13 or 19 lines (default 11); 2–8 seats (default 4); a human move deadline from 5 seconds to 64 minutes (default 10 seconds); a bot search time from 0.25 to 16 seconds (default 1 second); and a configurable ko rule: Auto (on for two players, off for three or more), Off, or On.
- **Seats and bots.** Room members who open a challenge may take one empty seat each before the start. Only the creator starts the game; every seat still empty becomes a server bot named `bot-`*seat*. A bot never moves for a person's seat, and a client cannot submit a bot's move.
- **Deadline.** An unanswered turn becomes a *timeout*: it passes the turn but neither ends nor resets the passing round. A move that arrives after its deadline is rejected and the timeout wins. A full round of timeouts abandons the game, scored as it stands.
- **Territory.** An empty region bordered only by one colour's stones and the Wall belongs to that colour. Score is territory minus losses.
- **Ending.** The game ends when every remaining player has passed since the last stone, after a full round of timeouts, or at $3N^2$ moves. A resigned seat passes automatically and cannot win. The server announces the final scores and winners in the game chat.
- **Ko.** When on, simple ko plus situational superko: a stone may not recreate an earlier board with the same seat to move. The Zobrist key only indexes candidates; equality is decided on the exact board and seat.
- **Deferred.** Team merges, handicaps, ratings, persistence, game record export and reconnect grace beyond reloading the page. Watchers may use game chat.

The positive invariants are that every board, clock and score shown comes from one server-authoritative transition, and that every turn and every game ends. Forbidden outcomes include a move for another seat or out of turn, two transitions for one turn (move versus deadline versus

bot result), a bot result applied after its turn ended, an outsider reading or changing a game, and unbounded bot computation. The implementation mapping binds bot results to the game and turn revision, runs searches outside the game lock with a time budget, a node cap and a bounded number of concurrent search slots, and covers each forbidden outcome with a race-enabled test.

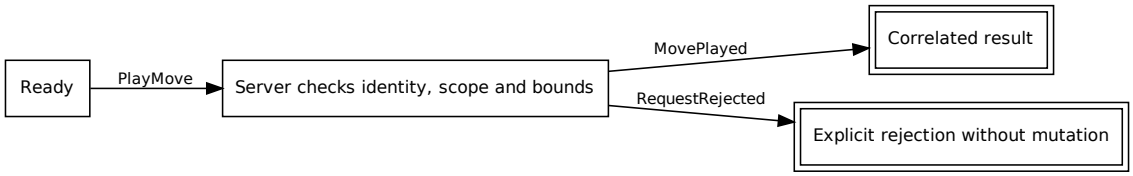


Figure 6.1: Play a stone, pass or resign: state transitions generated from the current typed task.

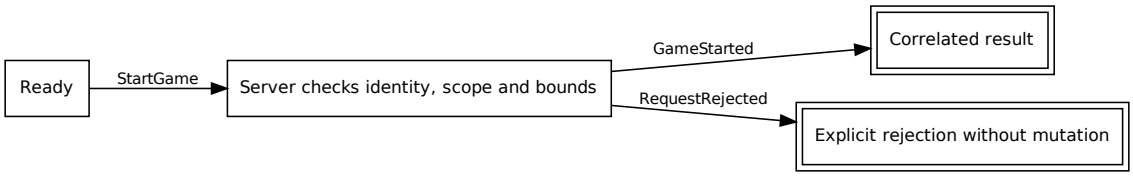


Figure 6.2: Start a game; bots fill empty seats: state transitions generated from the current typed task.

Chapter 7

Lab parity revision: seats, teams, history and search

7.1 Request

On October 26 the stakeholder asked that the server game behave like the N-Player Go Lab, the single-page prototype whose engine the server already ports. The important behaviours are: choosing which seats are bots and which are people; teams that people can merge; navigating the board history and playing out alternatives that are not the actual game; and seeing the Monte Carlo tree search estimations. The revision is also an exercise of the LeanFM method: book, typed grammar and tasks, two gates, then a one-shot implementation. Everything needed to continue lives in this book, `spec/`, `source/` and `reports/`; the chat that produced it is disposable.

7.2 Decisions recorded from the stakeholder

Four questions were put to the stakeholder; each answer is now a requirement.

1. **Seat control.** The creator plans each seat as *bot* or *open* before the start. A person who holds a seat may switch *autopilot* on (a server bot plays the seat) and off at any time during play; the seat stays theirs.
2. **Teams.** The creator may preset teams, as in the Lab roster. During play two teams merge when every person seated in either team accepts; bots go along with their team's people; merges are permanent.
3. **Analysis.** Watchers may branch and run local search at any time. Seated people may step through the actual moves during play, and may branch and search once the game has finished.
4. **Search visibility.** Each bot move publishes its search summary as soon as it is played.

7.3 Derived rules

- A seat's team defaults to its seat number, so everyone plays alone unless the creator groups seats. Team numbers run from 1 to the seat count.
- A team with no seated person cannot be party to a merge: nobody could consent for it. A merged team takes the smaller team number.

- A proposal is withdrawn by any decline, by any change to either team, or by the game finishing. Only one proposal per pair of teams is pending.
- Team score is the sum of its members' territory minus the sum of their losses. Winners are the team or teams with the highest team score among teams with at least one member who has not resigned. Bots on a team search for the team's combined score (the Lab's team reward).
- Autopilot moves use the bot deadline and publish search summaries. Switching autopilot off during the seat's own turn discards the pending bot result; the person then has the rest of the human deadline measured from the start of the turn.
- Each removed stone is recorded against the seat that captured it, giving the Lab's per-capturer *bowl*; suicide is recorded against the mover.
- Bot search settings are challenge parameters with the Lab's choices: exploration constant 0.5, 0.7 (default), 1.0 or 1.4; reward *margin* (default), *win* or *rank*; margin softness 2, 4 (default) or 8.

7.4 What every viewer sees

The browser reproduces the Lab against server state:

- the board with Wall coordinates, territory squares, the last-move marker, ghost markers for next moves, and a search heat overlay;
- navigation: Home, Back, Forward, End, previous and next variation, by buttons and by the arrow, Home and End keys; a variation tree; next-move chips;
- a *Search at this position* panel: candidate moves with visit share and reward for the mover, from the bot's published summary or the viewer's own local search;
- a per-seat table with Territory, Bowl, Score, Proj. (projected final score) and Outlook (expected reward), and team totals;
- a clickable projected-score timeline along the viewed line;
- GIF export of the viewed line; stone, capture and final-ten-seconds sounds with a toggle;
- live following while the viewer is at the latest move; otherwise a notice that new moves arrived.

Variations and local searches run in the viewer's browser with the Lab engine, are never sent to the server, and vanish when the page closes. This boundary is a fairness convention enforced by the provided browser, not a security boundary: anyone can copy a public position into another tool.

7.5 Protocol shape

New requests: `SetSeatPlan`, `LeaveSeat`, `SetAutopilot`, `ProposeMerge`, `RespondMerge`; `CreateGame` gains the bot settings and a seat plan; `GetGameState` gains `since_move` so a viewer fetches only new move records. Server-originated events (`BotMovePlayed`, `TurnTimedOut`, `AutoPassed`, `TeamsMerged`, `MergeWithdrawn`, `GameFinished`) reach game members as entries of `GameState`. A third actor, `Bot`, is the bounded pool of server search workers: the server sends `BotTurn` and accepts `BotMoveChosen` only for the same game, seat and turn revision before the deadline.

The grammar composes per-operation tasks into the `game_lifecycle` task: create, then any number of seat-plan, take, leave, join, chat and state operations, then start, then any number of turns, autopilot switches, merges, chats and state reads, then `GameFinished`. A `turn` is a person's move, a bot turn, a timeout or an automatic pass. Positive obligations: every turn ends exactly once; every started game finishes. Negative obligations cover moving for another seat, double-ended turns, stale bot results, merges without consent, analysis leaking into the game, and unbounded bot work.

7.6 Deferred

Handicaps, ratings, persistence across restarts, SGF export, push transport (the browser polls), reconnect grace beyond reloading the page, and moderation.

7.7 Implementation argument

The separate mapping in `spec/Implementation.lean` binds the `Bot` actor to a bounded worker pool, extends the Go engine's search to return the Lab's summary (top candidates by visits; per seat the mean reward and mean final score), keeps team arrays in the authoritative game, and serves the Lab engine and search worker as same-origin static files so the browser can branch and search locally under the existing content security policy (extended only with same-origin workers and `blob:`/`data:` images for GIF preview). Each negative obligation maps to a control and a race-enabled test listed in `source/traceability.json`.

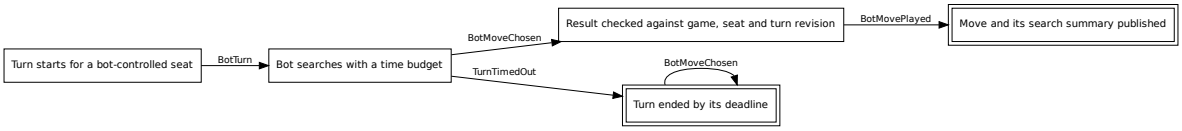


Figure 7.1: A server bot plays one turn: generated from the current typed task. A late result after the timeout loops on `timed_out` and is discarded.

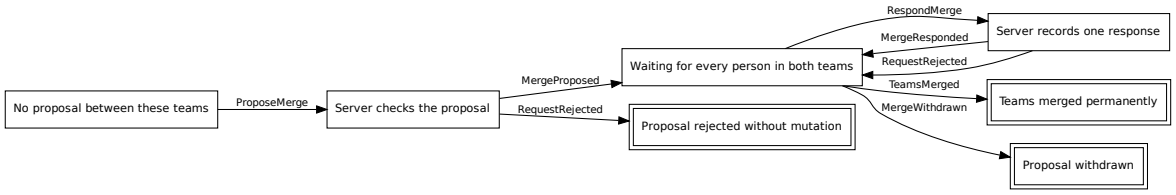


Figure 7.2: Two teams merge by consent: generated from the current typed task.

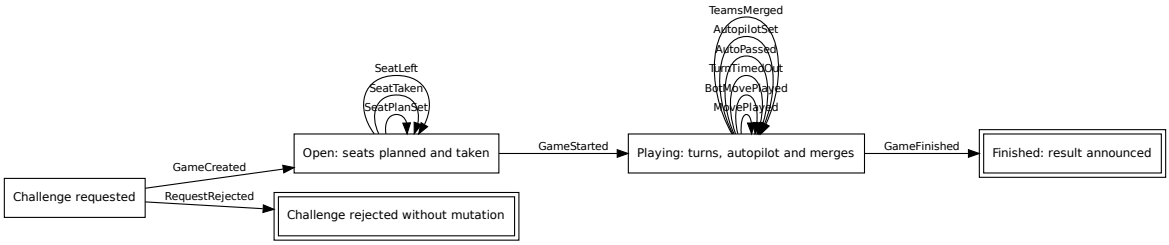


Figure 7.3: A game from challenge to result: generated from the current typed task.

Chapter 8

Usage scenarios

These screenshots come from the running application, driven by a script that plays the parts of three people: alice creates the challenge, bob joins her, and carol watches. `make book-tour` in `source/` reproduces them; `book/assets/tour/manifest.json` records each step, viewer, image hash, source hashes and browser version. Bot moves come from a time-limited search, so the stones differ between runs while the steps do not.

8.1 Welcome

A visitor chooses a local display name. Nothing else is needed for the local preview.

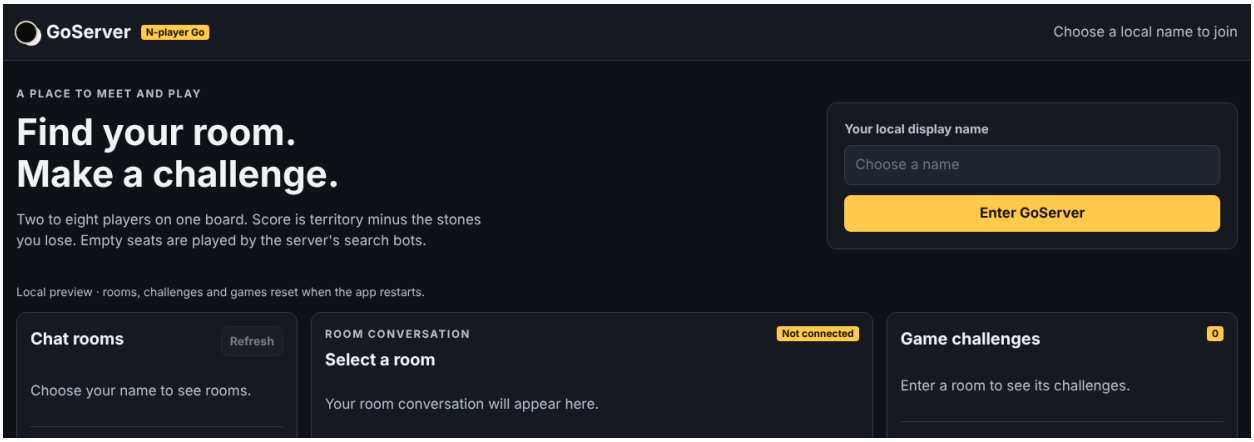


Figure 8.1: Welcome, seen by alice. Steps: open the app.

8.2 Creating a challenge

Alice is in the lobby and fills in a challenge: a 9×9 board, three players, a one-minute human deadline and half a second per bot move. The ko rule stays on Auto, which is off for three players.

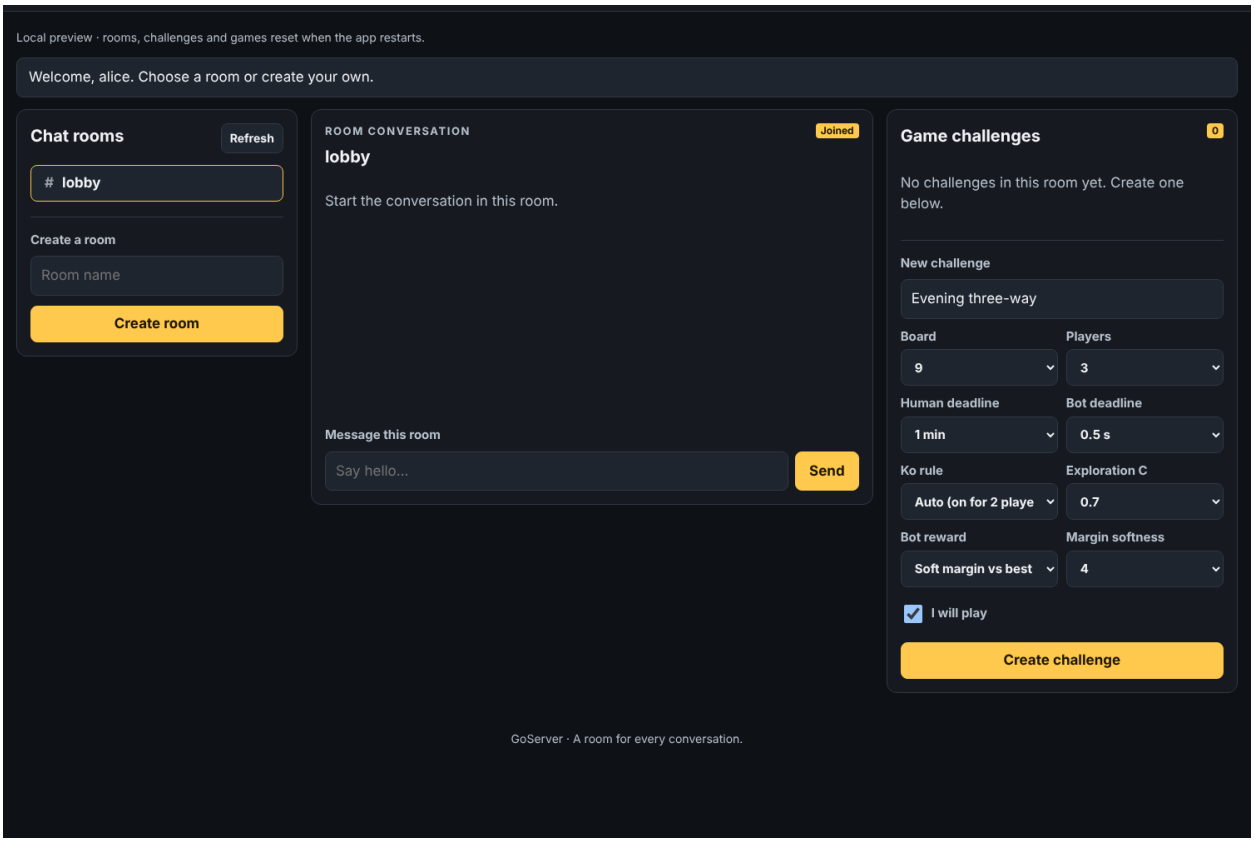


Figure 8.2: Creating a challenge, seen by alice. Steps: sign in as alice; fill in the New challenge form.

8.3 Planning seats

Before the start the creator decides, seat by seat, whether a person or a bot plays and which team the seat is on. Alice took seat 1, bob sat in seat 2, and alice marked seat 3 as a bot.

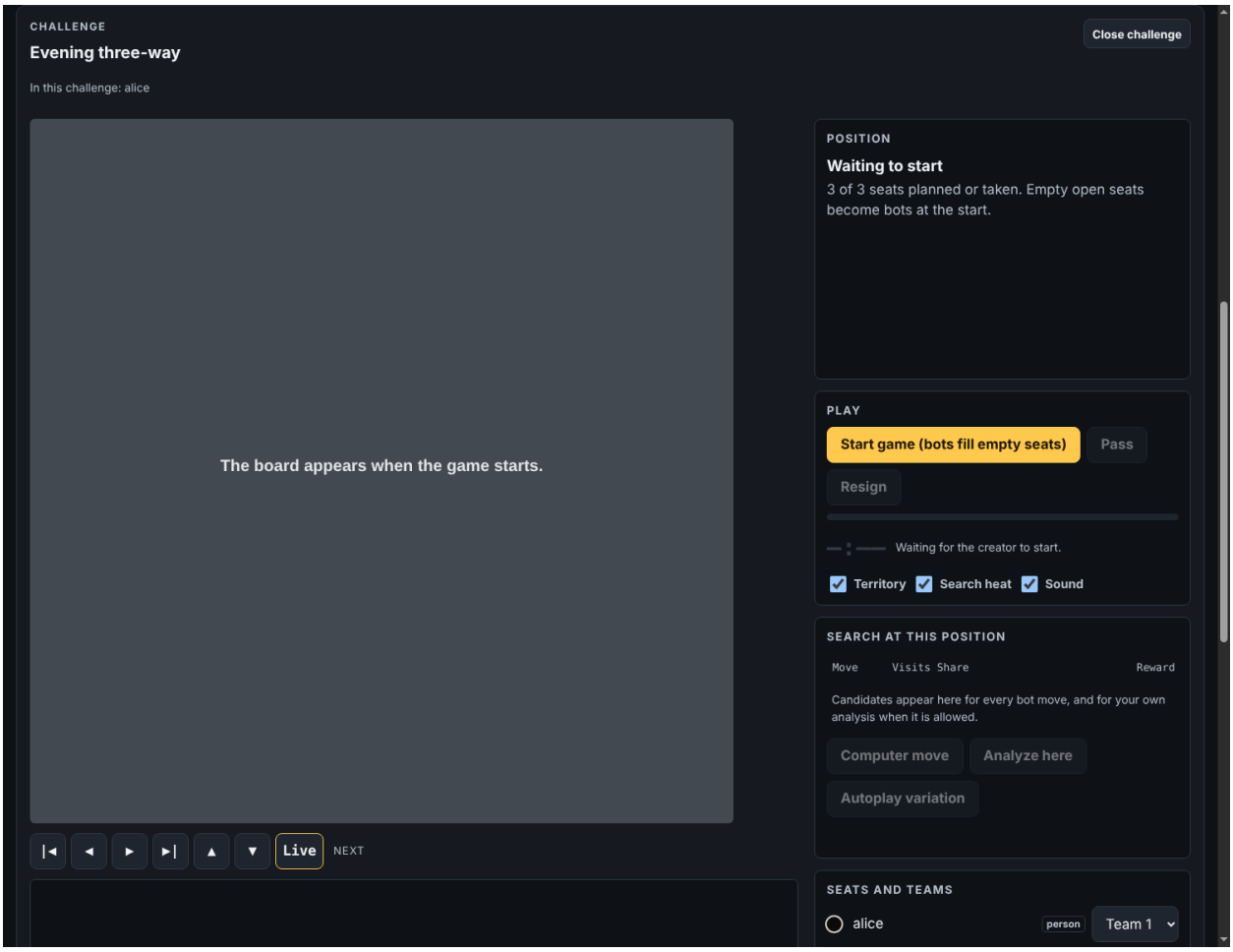


Figure 8.3: Planning seats, seen by alice. Steps: alice: create the challenge (seated automatically); alice: set seat 3 to Bot; bob: open the challenge, Sit here.

8.4 Reading the bot's search

Alice steps back one move, to the position where the bot searched for its last move. The panel lists the moves that search weighed, with visit share and expected reward; the blue discs on the board are the same search as a heat map. Proj. and Outlook give every seat's projected final score and expected reward, and the timeline under them plots the projections move by move.

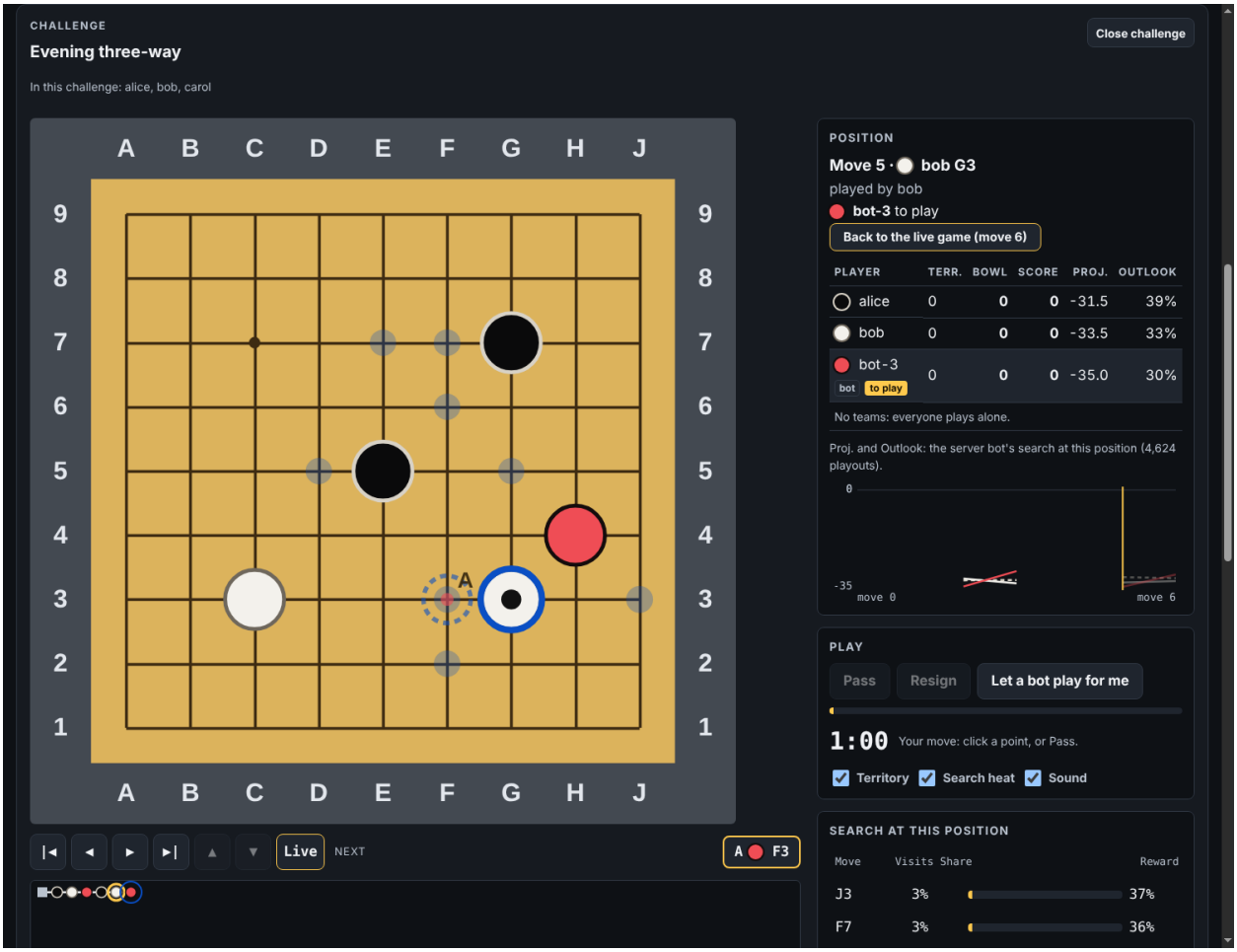


Figure 8.4: Reading the bot's search, seen by alice. Steps: alice and bob alternate stones by clicking the board; the bot answers each round; alice: Back one move.

8.5 A watcher explores a variation

Carol is watching, not playing. She steps back to move 3, tries a different stone (the gold ring marks a variation), and asks for a local search. The analysis runs in her browser only: the dashed branch in the tree never reaches the server, and the live game continues.

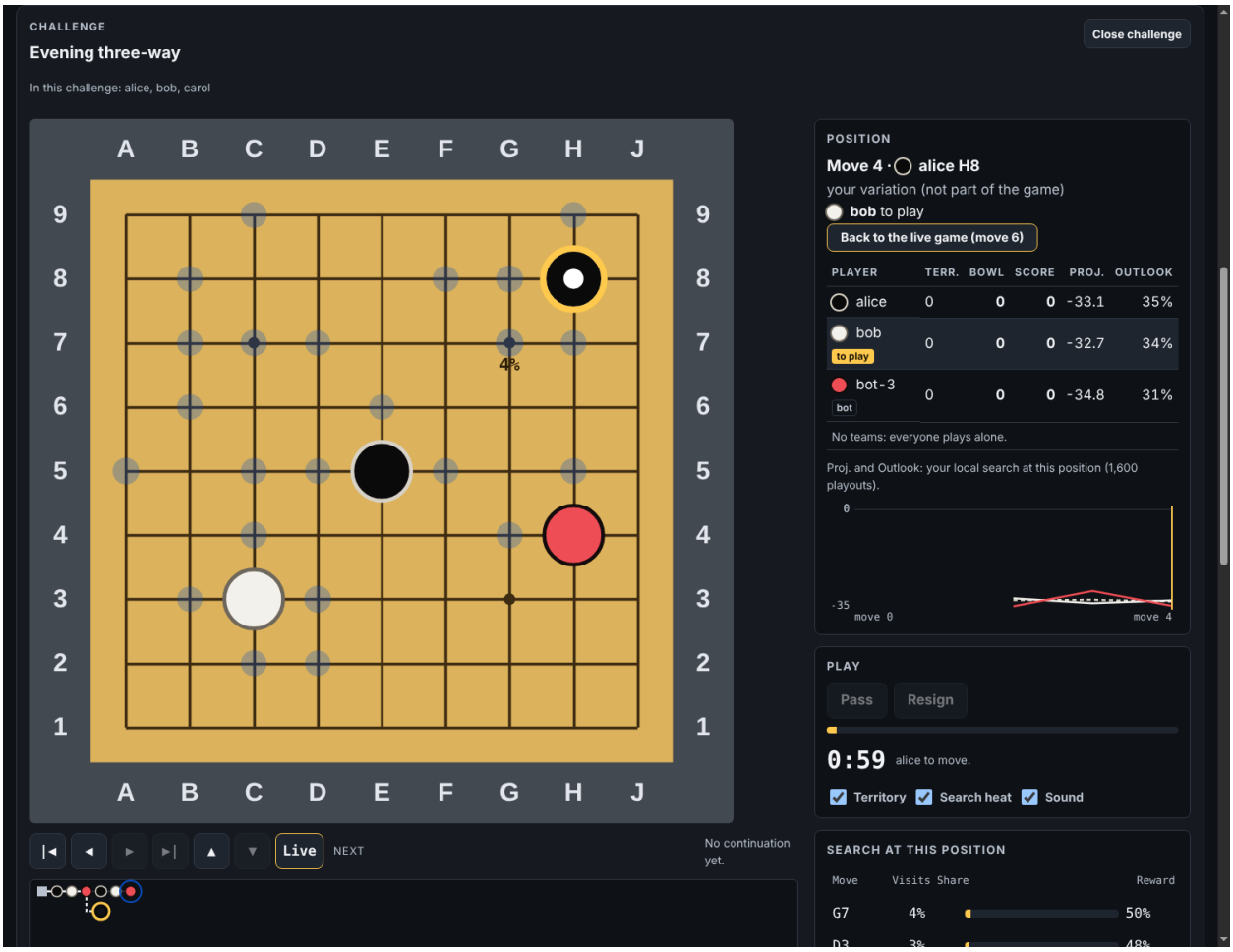


Figure 8.5: A watcher explores a variation, seen by carol. Steps: carol: open the challenge as a watcher; carol: Home, then Forward three times; carol: click an empty point; carol: Analyze here.

8.6 Proposing a team merge

During play bob proposes merging his team with alice's. The proposal shows who has accepted and who is still to answer; it takes effect only when every person on both teams accepts, and it is permanent.

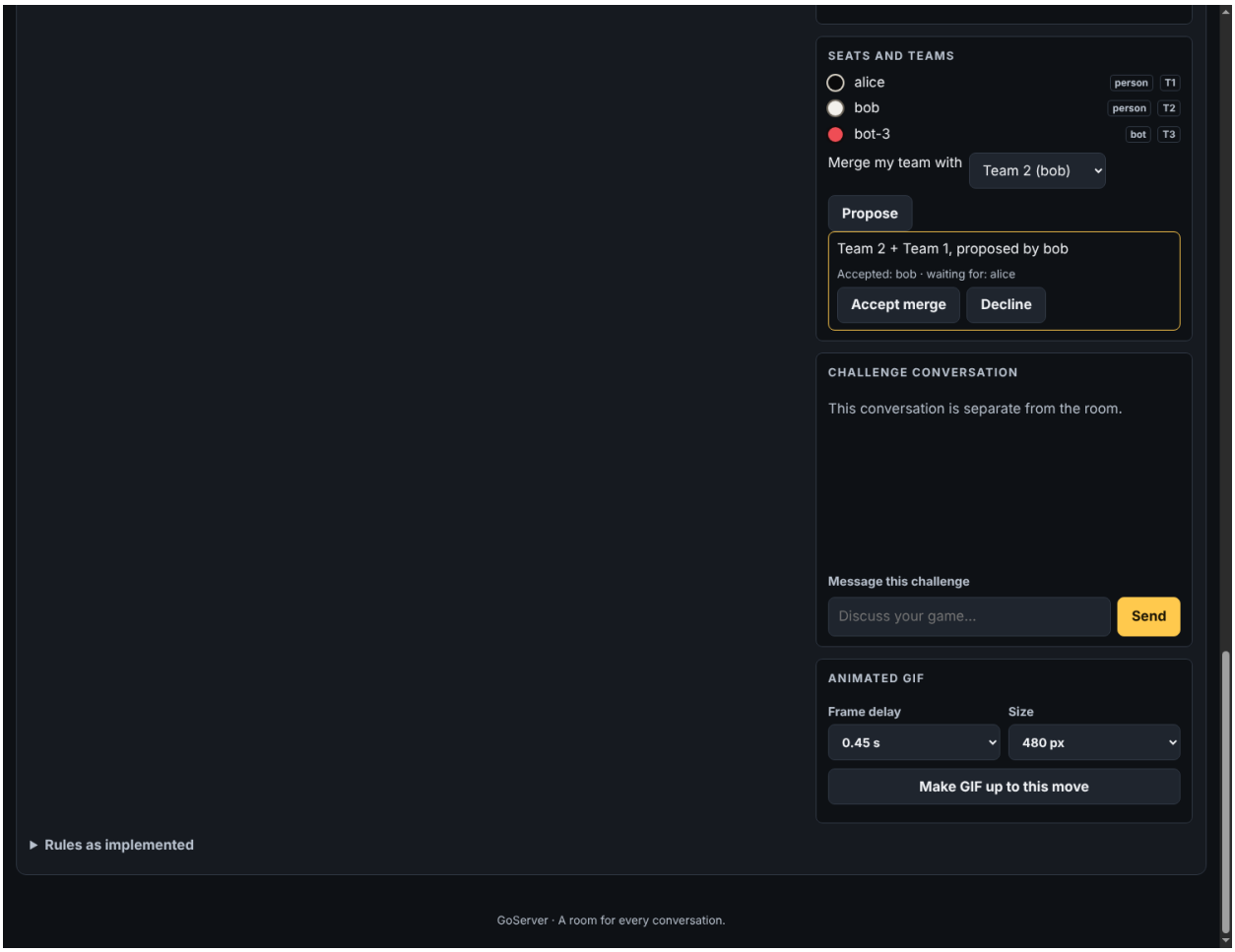


Figure 8.6: Proposing a team merge, seen by alice. Steps: bob: Merge my team with Team 1, Propose.

8.7 Playing as a team, on autopilot

After the merge the score table shows alice and bob as team 1 with a combined total. Alice then lets a bot play her seat; the seat, team and name stay hers, and the bot's move publishes its search like any other bot move. She can take the seat back at any time.

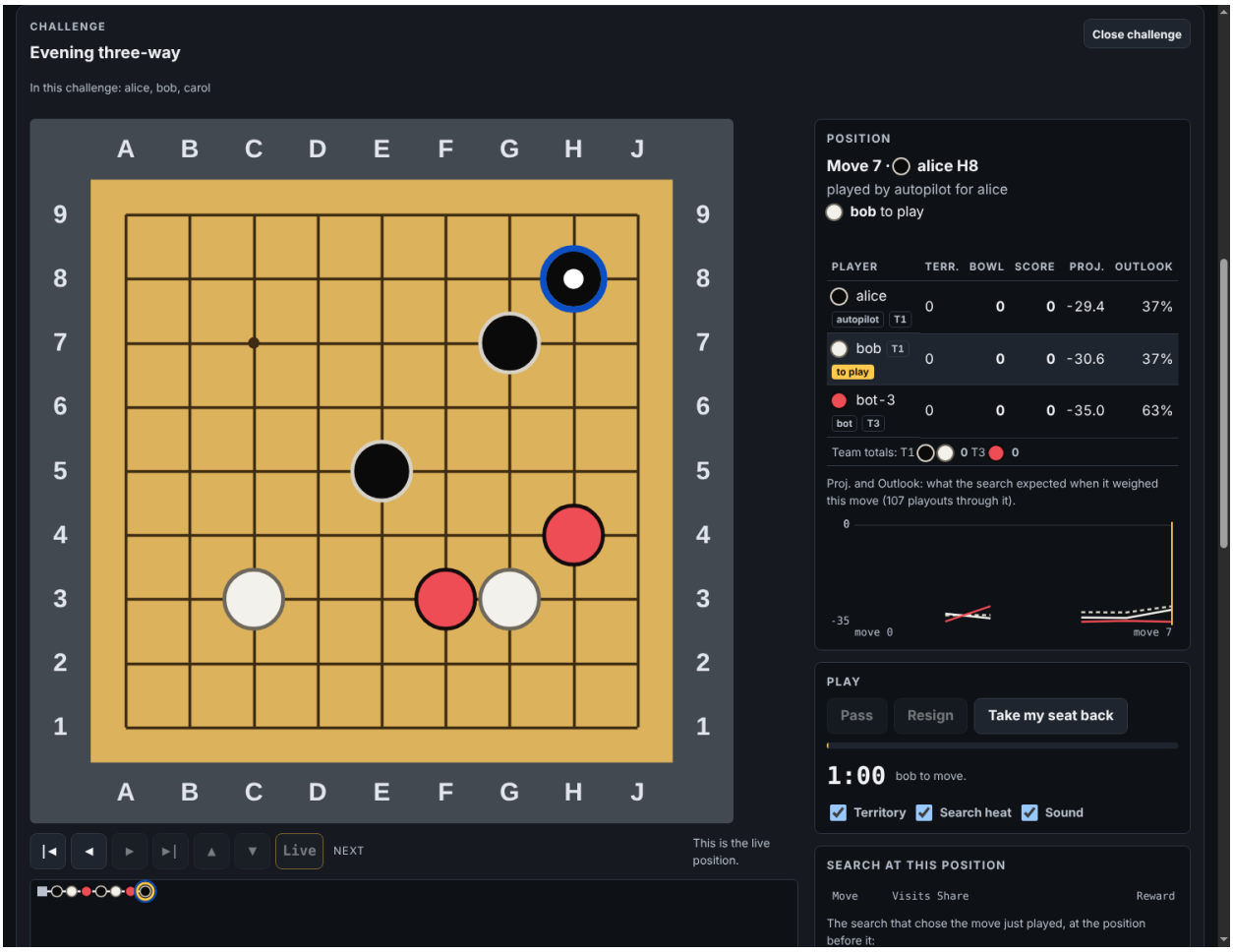


Figure 8.7: Playing as a team, on autopilot, seen by alice. Steps: alice: Accept merge; alice: Let a bot play for me.

8.8 The result

When every remaining player has passed in a row the game ends. The server announces team scores and the winners in the game conversation. Everyone, players included, can now branch and analyse any position.

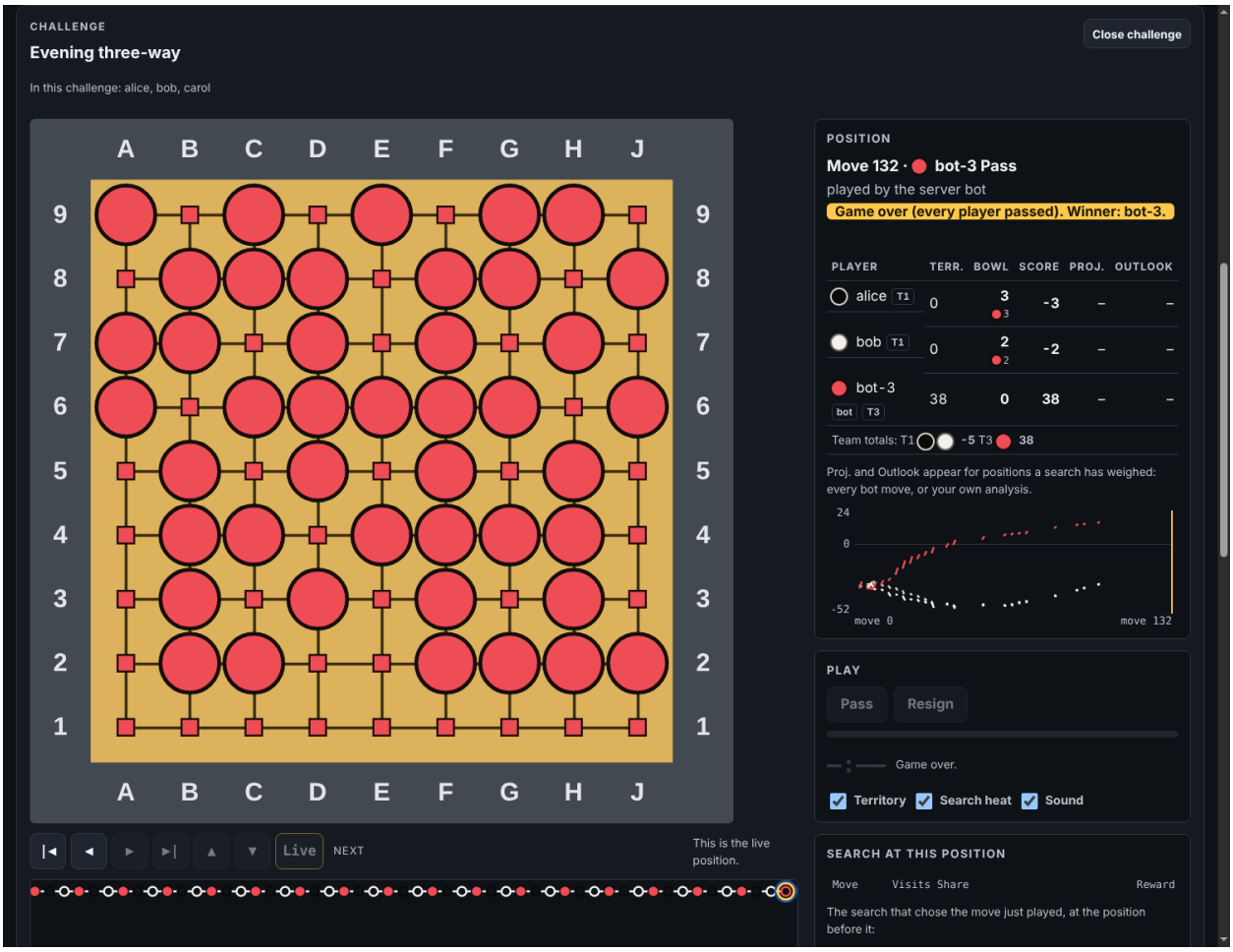


Figure 8.8: The result, seen by alice. Steps: alice and bob pass until the bot also passes.